Python3-

基础教程

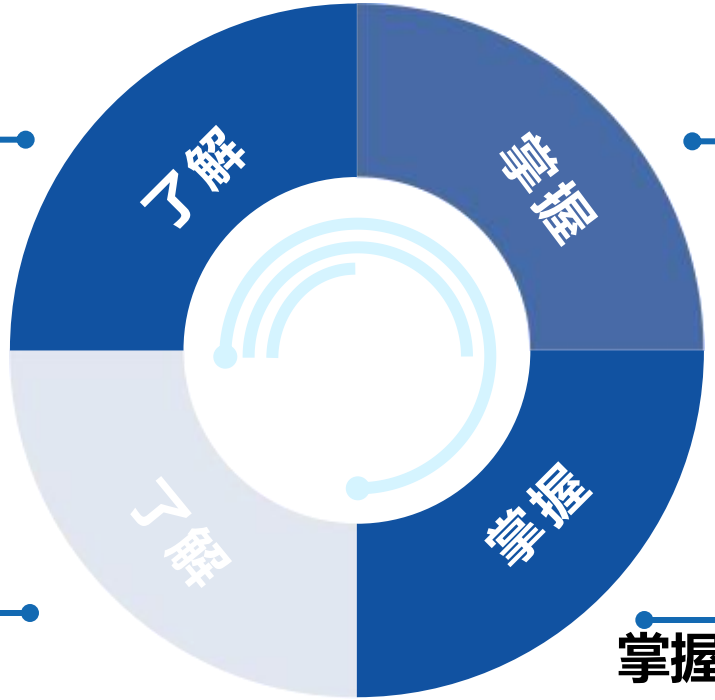
第2章 Python语法基础

- 变量和变量类型
- 标识符和关键字
- 简单数值类型
- 运算符

学习目标

1 了解Python中的关键字

2 掌握Python的变量和变量类型



3 掌握Python中的标识符，可以识别合法的标识符

4 了解Python不同运算符的使用

目录页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

注释

Python中的单行注释以#开头！

```
# 第一个注释
print (“Hello, Python!”)  # 第二个注释
```

注释

多行注释可以使用三引号作为开头和结束符号

```
"""
print(value, ..., sep=' ', end='\n',
file=sys.stdout, flush=False)
"""
```

行与缩进

python最具特色的就是使用缩进来表示代码块

```
if True:
        print ("True")
else:
        print ("False”)
        print ("False”)
```

```
if True:
        print ("True")
else:
        print ("False")
  print (”hello”)
```

语句换行

Python 通常是一行写完一条语句，但如果语句很长，我们需要换行，这时可以使用圆括号来实现。

```
str = ('Python是一种面向对象、解释型计算机程序设计语言，'
       '由Guido van Rossum于1989年底发明。'
       '第一个公开发行版发行于1991年，'
       '源代码同样遵循 GPL(GNU General Public License)协议。')
```

语句换行

Python 通常是一行写完一条语句，但如果语句很长，我们需要换行，这时可以使用\来实现。

```
str = 'Python是一种面向对象、解释型计算机程序设计语言，' \
      '由Guido van Rossum于1989年底发明。' \
      '第一个公开发行版发行于1991年， ' \
      '源代码同样遵循 GPL(GNU General Public License)协议。'
```

语句换行

需要注意的是，在 [], {}, 或 () 中的语句，不需要使用圆括号进行换行。

```
total = ['item_one', 'item_two', 'item_three',
         'item_four', 'item_five']
```

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

变量和赋值

超市购物

现实生活中，大家去超市买东西的时候，往往都需要一个菜篮子，用来进行存储物品，等到所有的物品都购买完成后，在收银台进行结账即可。

变量和赋值

Python中的变量用来存储数据，变量可以理解为去超市购物的菜篮子，其类型和值在赋值的那一刻被初始化。

```
num1 = 100
num2 = 87
result = num1 + num2
```

num1和num2变量就好比一个小菜篮子，它们分别存储的数据是100和87。result变量存储的数据是num1和num2这两个“菜篮子”中的数据累计之和。

变量的类型

1. 数字类型
   - 整型
   - 复数
   - 浮点型
2. 布尔类型
3. 字符串类型
4. 列表类型
5. 元组类型
6. 字典类型
7. 集合

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

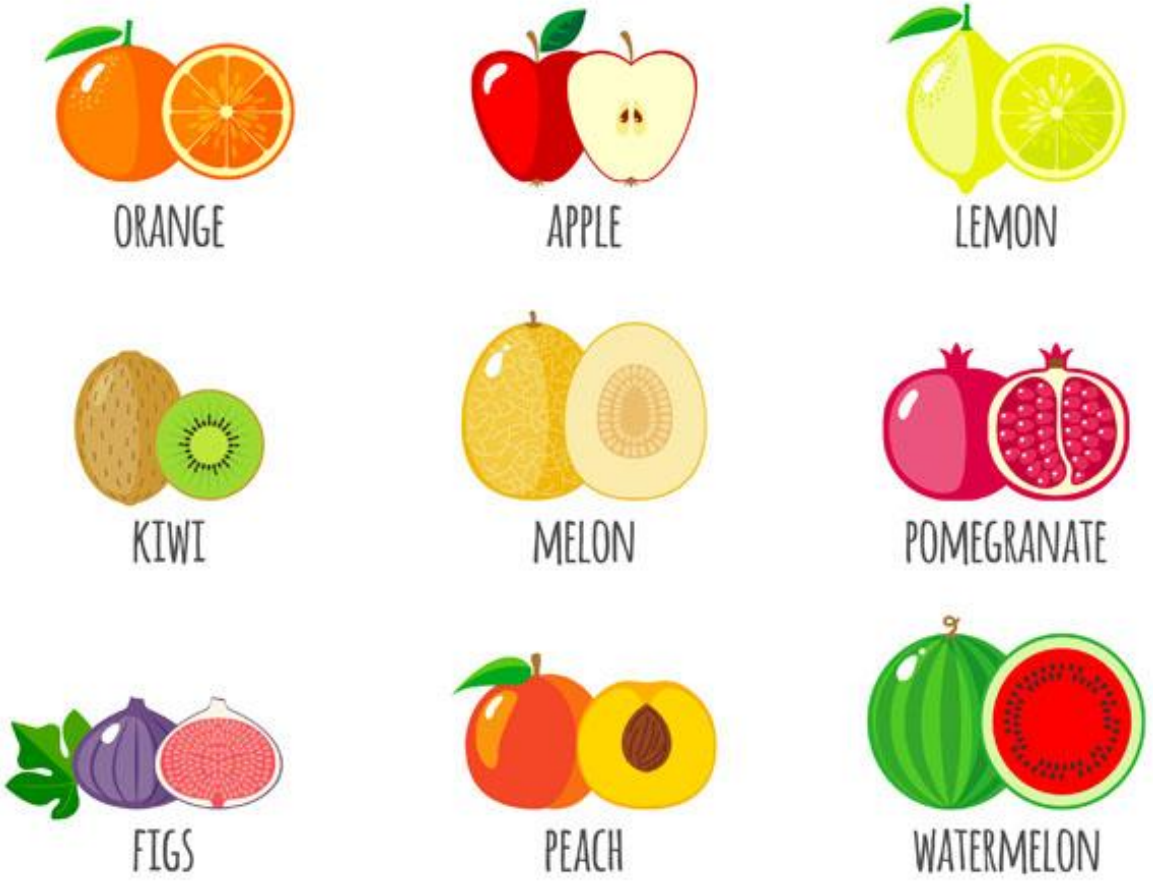
06 位运算

07 运算符优先级

标识符

水果名称

现实生活中，人们常用一些名称来标记事物，例如，现实生活中每种水果都有一个名称来标识。



标识符

若希望在程序中表示一些事物，需要开发人员自定义一些符号和名称，这些符号和名称叫做**标识符。**

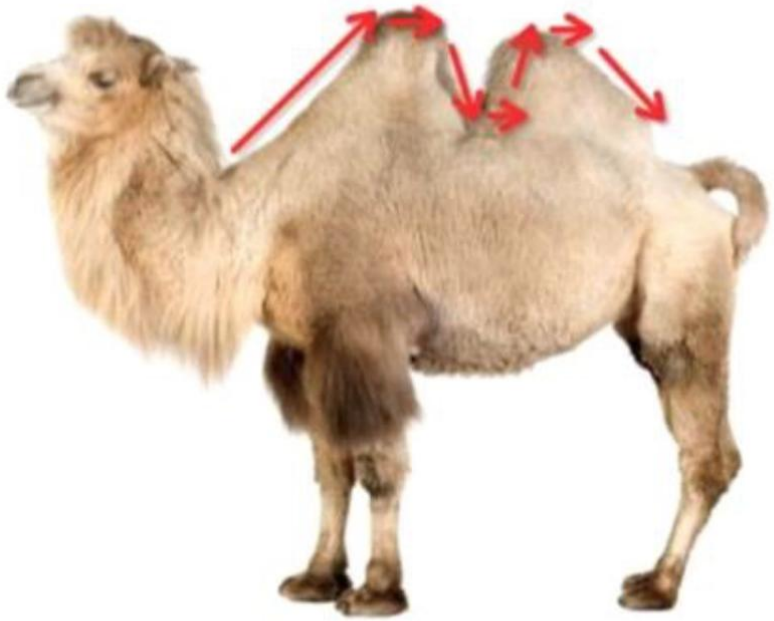
命名规则

- 标识符由字母、下划线和数字组成，且数字不能开头。
- Python中的标识符是区分大小写的。
- python中的标示符不能使用关键字

标识符

为了规范命名标识符，关于标识符的命名提以下建议：

1. 见名之意
2. 驼峰式



如： userName　　userLoginFlag

关键字

关键字指的是具有特殊功能的标识符。

```
>>> help()              # 进入帮助系统
help> keywords          # 查看所有的关键字列表
help> return            # 查看return这个关键字的说明
help> quit              # 退出帮助系统
```

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

整型

整数类型（int）简称整型，它用于表示整数。

– 二进制：0b10100

–八进制: 0o10111

–十进制: 64

–十六进制: 0x14

浮点型

- 浮点型（Float）用于表示实数。
- 浮点型字面值可以用十进制或科学计数法表示。

<实数>E或者e<整数>

E或e表示基是10，后面的整数表示指数，指数的正负使用+或-表示。

布尔类型

- 布尔型数据只有两个取值：True和False
- bool值没有操作
- 所有的空都是False，所有的非空都是True

```
a = 0 print(bool(a)) # 整数0,False
s = "" print(bool(s)) # 空字符串，False
lst = [] print(bool(lst)) # 空列表，False
dic = {} print(bool(dic)) # 空字典，False
a = None print(bool(a))# None表示空，False
```

布尔类型

– 以下对象的布尔值都是False：

- None
- False（布尔型）
- 0（整型0）
- 0.0（浮点型0）
- 0.0+0.0j（复数0）
- “”（空字符串）
- []（空列表）
- ()（空元组）
- {}（空字典）

复数类型

复数类型，用于表示数学中的复数，例如，5+3j。

两大特点

（1）复数由实数部分和虚数部分构成，表示为：real+imagj 或 real+imagJ

（2）复数的实数real和虚数imag都是浮点型

数字类型转换

函数	说明
int(x [,base])	将x转换为一个整数
float(x)	将x转换到一个浮点数，默认六位小数
complex(real [,imag])	创建一个复数

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

算术运算符

运算符	相关说明
+	加：两个对象相加
-	减：得到负数或一个数减去另一个数
*	乘：两个数相乘或是返回一个被重复若干次的字符串
/	除：x除以y
%	取余：返回除法的余数
**	幂：返回x的y次幂
//	取整除：返回商的整数部分

赋值运算符

赋值运算符只有一个，即=，它的作用是把等号右边的值赋给左边。 例如，x=1

```
anInt=12
anFloat=2.2
anStr='string'
aList=['a','a','a']
anArray=(1,2,3)
aMap={1:'a',2:'b',3:'c',}
```

为多个变量赋同一个值：x=y=z=1

增量赋值

```
x+=1  x=x+1
```

将多个值赋值给多个变量

```
a, b = 1, 2
```

变量交换

```
x = 2
y = 3
x,y = y,x
```

复合赋值运算符

运算符	相关说明	实例
+=	加法赋值运算符	c+=a等效于c=c+a
-=	减法赋值运算符	c-=a等效于c=c-a
*=	乘法赋值运算符	c*=a等效于c=c*a
/=	除法赋值运算符	c/=a等效于c=c/a
%=	取模赋值运算符	c%=a等效于c=c%a
**=	幂赋值运算符	c**=a等效于c=c**a
//=	取整除赋值运算符	c//=a等效于c=c//a

复合赋值运算符

```
a = 21
b = 10
c = 0

c = a + b
print ("1 - c 的值为: ", c)

c += a
print ("2 - c 的值为: ", c)

c *= a
print ("3 - c 的值为: ", c)

c /= a
print ("4 - c 的值为: ", c)

c = 2
c %= a
print ("5 - c 的值为: ", c)

c **= a
print ("6 - c 的值为: ", c)

c //= a
print ("7 - c 的值为: ", c)
```

```
1 - c 的值为:  31
2 - c 的值为:  52
3 - c 的值为:  1092
4 - c 的值为:  52.0
5 - c 的值为:  2
6 - c 的值为:  2097152
7 - c 的值为:  99864
```

比较运算符

运算符	相关说明
==	检查两个操作数的值是否相当
!=	检查两个操作数的值是否相等
>	检查左操作数的值是否大于右操作数的值
<	检查左操作数的值是都小于右操作数的值
>=	检查左操作数的值是否大于或等于右操作数的值
<=	检查左操作数的值是否小于或等于右操作数的值

逻辑运算符

运算符	逻辑表达式	描述
and	x and y	布尔“与”，如果x为False，x and y返回False，否则它返回y的计算值
or	x or y	布尔“或”，如果x为True，它返回True，否则返回y的计算值
not	not x	布尔“非”，如果x为True，返回False，如果x为False，它返回True

逻辑运算符

```python
a = 10
b = 20

if ( a and b ):
   print ("1 - 变量 a 和 b 都为 true")
else:
   print ("1 - 变量 a 和 b 有一个不为 true")

if ( a or b ):
   print ("2 - 变量 a 和 b 都为 true，或其中一个变量为 true")
else:
   print ("2 - 变量 a 和 b 都不为 true")
```

```python
a = 0
if ( a and b ):
   print ("3 - 变量 a 和 b 都为 true")
else:
   print ("3 - 变量 a 和 b 有一个不为 true")

if ( a or b ):
   print ("4 - 变量 a 和 b 都为 true，或其中一个变量为 true")
else:
   print ("4 - 变量 a 和 b 都不为 true")

if not( a and b ):
   print ("5 - 变量 a 和 b 都为 false，或其中一个变量为 false")
else:
   print ("5 - 变量 a 和 b 都为 true")
```

```
1 - 变量 a 和 b 都为 true
2 - 变量 a 和 b 都为 true，或其中一个变量为 true
3 - 变量 a 和 b 有一个不为 true
4 - 变量 a 和 b 都为 true，或其中一个变量为 true
5 - 变量 a 和 b 都为 false，或其中一个变量为 false
```

成员运算符

运算符	描述	实例
in	如果在指定的序列中找到值返回 True，否则返回 False。	x 在 y 序列中，如果 x 在 y 序列中返回 True。
not in	如果在指定的序列中没有找到值返回 True，否则返回 False。	x 不在 y 序列中，如果 x 不在 y 序列中返回 True。

成员运算符

```
a = 10
b = 20
list = [1, 2, 3, 4, 5 ];

if ( a in list ):
   print ("1 - 变量 a 在给定的列表中 list 中")
else:
   print ("1 - 变量 a 不在给定的列表中 list 中")

if ( b not in list ):
   print ("2 - 变量 b 不在给定的列表中 list 中")
else:
   print ("2 - 变量 b 在给定的列表中 list 中")

# 修改变量 a 的值
a = 2
if ( a in list ):
   print ("3 - 变量 a 在给定的列表中 list 中")
else:
   print ("3 - 变量 a 不在给定的列表中 list 中")
```

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

位运算符

运算符	描述
&	按位与运算符：参与运算的两个值,如果两个相应位都为1,则该位的结果为1,否则为0
|	按位或运算符：只要对应的二个二进位有一个为1时，结果位就为1。
^	按位异或运算符：当两对应的二进位相异时，结果为1
~	按位取反运算符：对数据的每个二进制位取反,即把1变为0,把0变为1。`~x` 类似于 `-x-1`
<<	左移动运算符：运算数的各二进位全部左移若干位，由"<<"右边的数指定移动的位数，高位丢弃，低位补0。
>>	右移动运算符：把">>"左边的运算数的各二进位全部右移若干位，">>"右边的数指定移动的位数

按位与

参与运算的两个数各对应的二进位进行“与”的操作。只有对应的两个二进位都是1时，结果位就为1，否则结果位为0

	0	0	0	0	1	0	0	1
按位与（&）	0	0	0	0	0	0	1	1
运算结果	0	0	0	0	0	0	0	1

按位或

按位或指的是参与运算的两个数各对应的二进位进行“或”的操作。只要对应的两个二进位有一个为1时，结果位就为1

	0	0	0	0	1	0	0	0
按位或（|）	0	0	0	0	0	0	1	1
运算结果	0	0	0	0	1	0	1	1

按位异或

- 按位异或就是将参与运算的两个数对应的二进制位进行比较，如果一个位为1，另一个位为0，则结果为就为1，否则，结果位为0。

	0	0	0	0	1	0	0	0
按位异或（^）	0	0	0	0	0	1	0	0
运算结果	0	0	0	0	1	1	0	0

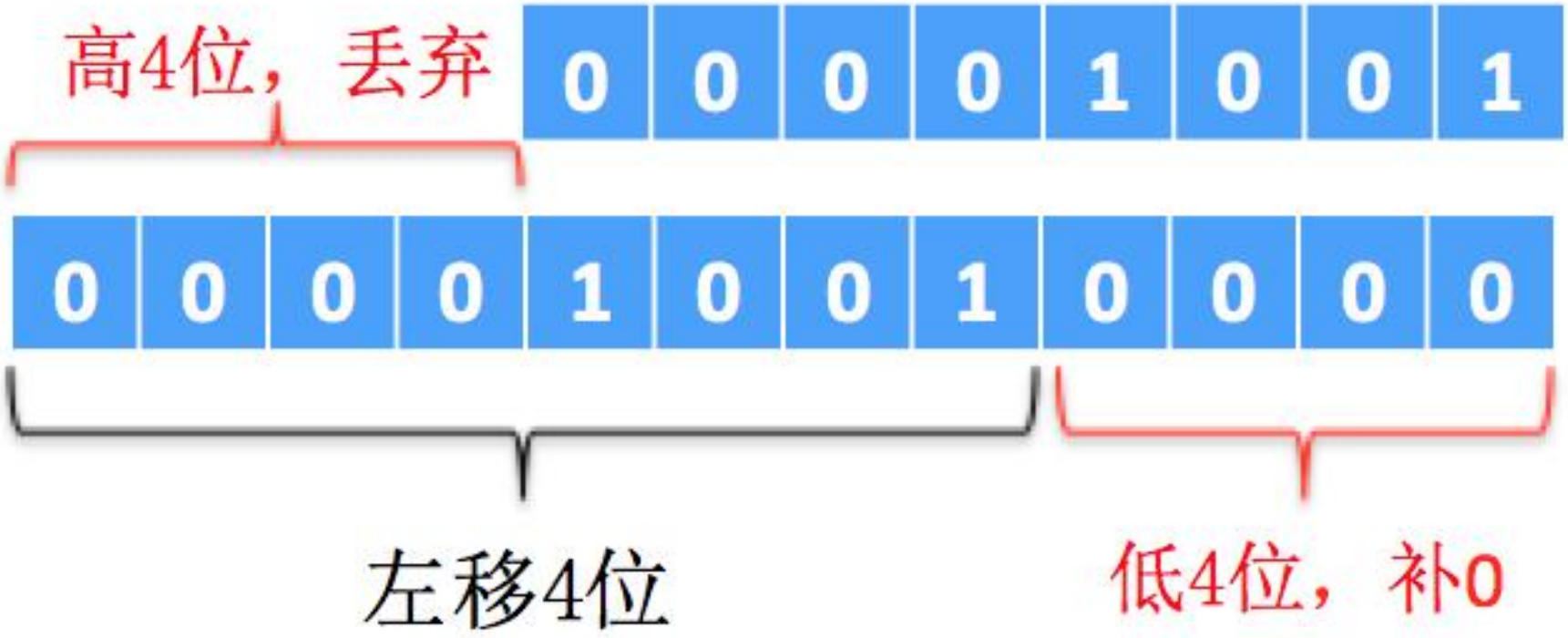
按位取反

- 按位取反就是将二进位的每一位进行取反；
- 0取反为1，1取反为0

9的二进制	0	0	0	0	1	0	0	1
按位取反	1	1	1	1	0	1	1	0

按位左移

按位左移指的是二进位全部左移n位，高位丢弃，低位补0 。



按位右移

按位右移指的是将二进位全部右移n位，移出的位丢弃，移进的位补符号位。

位运算

```
a = 60              # 60 = 0011 1100
b = 13              # 13 = 0000 1101
c = 0

c = a & b;          # 12 = 0000 1100
print ("1 - c 的值为: ", c)

c = a | b;          # 61 = 0011 1101
print ("2 - c 的值为: ", c)

c = a ^ b;          # 49 = 0011 0001
print ("3 - c 的值为: ", c)

c = ~a;             # -61 = 1100 0011
print ("4 - c 的值为: ", c)

c = a << 2;         # 240 = 1111 0000
print ("5 - c 的值为: ", c)

c = a >> 2;         # 15 = 0000 1111
print ("6 - c 的值为: ", c)
```

```
1 - c 的值为:  12
2 - c 的值为:  61
3 - c 的值为:  49
4 - c 的值为:  -61
5 - c 的值为:  240
6 - c 的值为:  15
```

过渡页

01 基本语法

02 变量和数据类型

03 标识符和关键字

04 简单数值类型

05 运算符

06 位运算

07 运算符优先级

运算符优先级

运算符	描述
**	指数（最高优先级）
~ + -	按位翻转，一元加号和减号 (最后两个的方法名为 +@ 和 -@)
* / % //	乘，除，取模和取整除
+ -	加法减法
>> <<	右移，左移运算符
&	位 'AND'
^ |	位运算符
<= < > >=	比较运算符
<> == !=	等于运算符
= %= /= //= -= += *= **=	赋值运算符
is is not	身份运算符
not or and	逻辑运算符

假设：

a = 20， b = 10， c = 15

d = 5， e = 0

那么

e = (a + b) * c / d=90

e = ((a + b) * c) / d=90

e = (a + b) * (c / d)=90

e = a + (b * c) / d =50

本章小结

- 本章主要讲解了Python中的变量、数据类型以及运算符，这种知识都是最基础的语法，也比较容易理解。希望大家在初学Python时期，多动手写代码进行练习，加深印象，为后期的深入学习打好扎实功底。